Reporte Proyecto 2ª Evaluación

Creación de los nodos necesarios

Para este proyecto se usó el archivo docker-compose de la actividad previamente realizada (Actividad 1 MongoDB cluster). Este archivo tiene todo lo necesario para levantar todos los contenedores de esta pequeña base de datos distribuida. Dentro de los servicios declarados tenemos:

- 1 servidor de configuración: encargado de almacenar los metadatos del clúster.
- 3 shards: nodos sobre los cuales se distribuirán los documentos.
- Mongos: el enrutador.

Archivo docker-compose:

```
services:
  configsvr:
    image: mongo:7.0
    command: mongod --configsvr --replSet configReplSet --port 27017 --
bind_ip_all
    container_name: configsvr
    ports:
      - "27017:27017"
    volumes:
      - configsvr:/data/db
    networks:
      - mongo-network

  shard1:
    image: mongo:7.0
    command: mongod --shardsvr --replSet shard1ReplSet --port 27018 --
bind_ip_all
    container_name: shard1
    ports:
      - "27018:27018"
    volumes:
      - shard1:/data/db
    networks:
      - mongo-network

  shard2:
    image: mongo:7.0
```

```
    command: mongod --shardsvr --replSet shard2ReplSet --port 27019 --
bind_ip_all
    container_name: shard2
    ports:
      - "27019:27019"
    volumes:
      - shard2:/data/db
    networks:
      - mongo-network

  shard3:
    image: mongo:7.0
    command: mongod --shardsvr --replSet shard3ReplSet --port 27020 --
bind_ip_all
    container_name: shard3
    ports:
      - "27020:27020"
    volumes:
      - shard3:/data/db
    networks:
      - mongo-network

  mongos:
    image: mongo:7.0
    container_name: mongos
    depends_on:
      - configsvr
      - shard1
      - shard2
      - shard3
    command: mongos --configdb configReplSet/configsvr:27017 --
bind_ip_all --port 27021
    ports:
      - "27021:27021"
    networks:
      - mongo-network

volumes:
  configsvr:
  shard1:
  shard2:
  shard3:

networks:
  mongo-network:
    driver: bridge
```

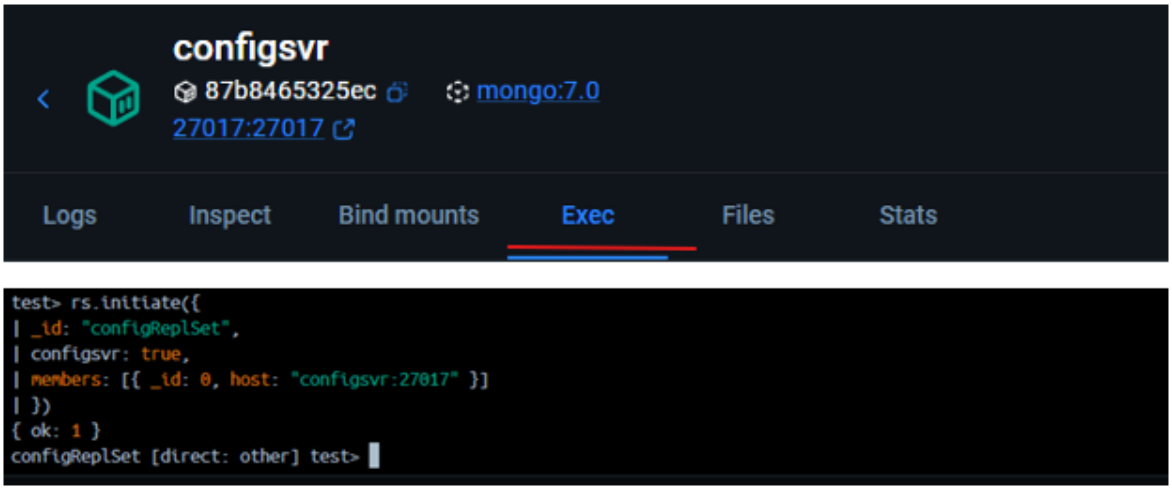
Ejecutar docker compose

Una vez hecho docker compose sobre nuestro archivo se generaron correctamente todos los servicios. Pudimos corroborar esto fácilmente al entrar a Docker Desktop donde se listan los contenedores.



Config-Server

Dentro de la consola de mongosh de nuestro servidor de configuración ejecutamos este pequeño script de configuración. Cómo se puede observar indicamos con la opción configsvr: true que esta instancia de mongo se trata del servidor de configuración.



Inicializar los shards

Para configurar los shard, accedimos a la terminal de mongosh de cada uno de ellos y ejecutamos el siguiente script. En la imagen se muestra un ejemplo del procedimiento realizado para configurar el shard1. La configuración de los otros dos shards se llevó a cabo siguiendo el mismo procedimiento.

```
# mongosh --port 27018
Current Mongosh Log ID: 6ae2bfddf6025ca1e8f38d6f
Connecting to:          mongodb://127.0.0.1:27018/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0
```

```
test> rs.initiate({
| _id: "shard1ReplSet",
| members: [{ _id: 0, host: "shard1:27018" }]
| })
{ ok: 1 }
shard1ReplSet [direct: other] test>
```

Agregar los shards a mongos

La instancia de mongos quedo en el puerto 27021, así que como vemos en la primera imagen accedimos a mongosh por este puerto para acceder a la terminal. Luego se procedió a agregar cada uno de los shards:

```
# mongosh --port 27021
Current Mongosh Log ID: 6ae2c22942e4712c6fde207d
Connecting to:          mongodb://127.0.0.1:27021/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.14.0
Using MongoDB:          7.0.45
Using Mongosh:          2.14.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/
```

```
[direct: mongos] test> sh.addShard("shard1ReplSet/shard1:27018")
{
  shardAdded: 'shard1ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793245833, i: 6 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793245833, i: 6 })
}
```

```
[direct: mongos] test> sh.addShard("shard2ReplSet/shard2:27019")
{
  shardAdded: 'shard2ReplSet',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793245881, i: 14 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793245881, i: 4 })
}
```

Verificar los shards agregados a mongos

Para verificar que los shards se agregaron con éxito a mongos se ejecutó el comando sh.status(), que como se muestra en la imagen nos lista los shards que añadimos en el apartado anterior.

```
[direct: mongos] test> sh.status()
shardingVersion
{ _id: 1, clusterId: ObjectId('6ae2bf148d14ff2c42580579') }
---
shards
[
  {
    _id: 'shard1ReplSet',
    host: 'shard1ReplSet/shard1:27018',
    state: 1,
    topologyTime: Timestamp({ t: 1793245833, i: 3 })
  },
  {
    _id: 'shard2ReplSet',
    host: 'shard2ReplSet/shard2:27019',
    state: 1,
    topologyTime: Timestamp({ t: 1793245881, i: 2 })
  },
  {
    _id: 'shard3ReplSet',
    host: 'shard3ReplSet/shard3:27020',
    state: 1,
    topologyTime: Timestamp({ t: 1793245913, i: 2 })
```

Crear la base de datos

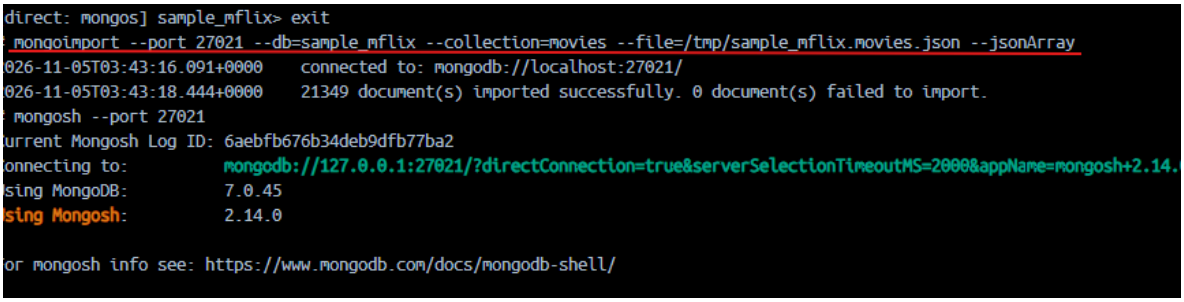
Para crear la base de datos con la semilla proporcionada "**sampe_mflix.movies**" primero movimos este archivo dentro del contendor con ayuda del siguiente comando:

```
docker cp sample_mflix.movies.json mongos:/tmp/sample_mflix.movies.json
```

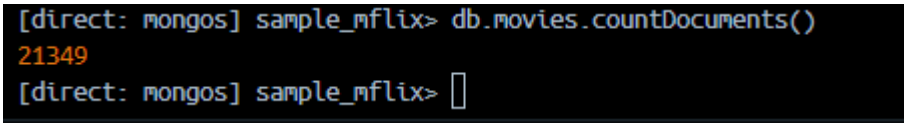
Pudimos confirmar que el archivo se movió correctamente al entrar al contendor desde Docker Desktop:



Luego, con el archivo ya adentro del contendor, se ejecutó el siguiente comando para poder importar la base de datos:



Al ejecutar el comando para contar los documentos pudimos ver que los 21340 documentos se agregaron sobre correctamente sobre la colección movies.



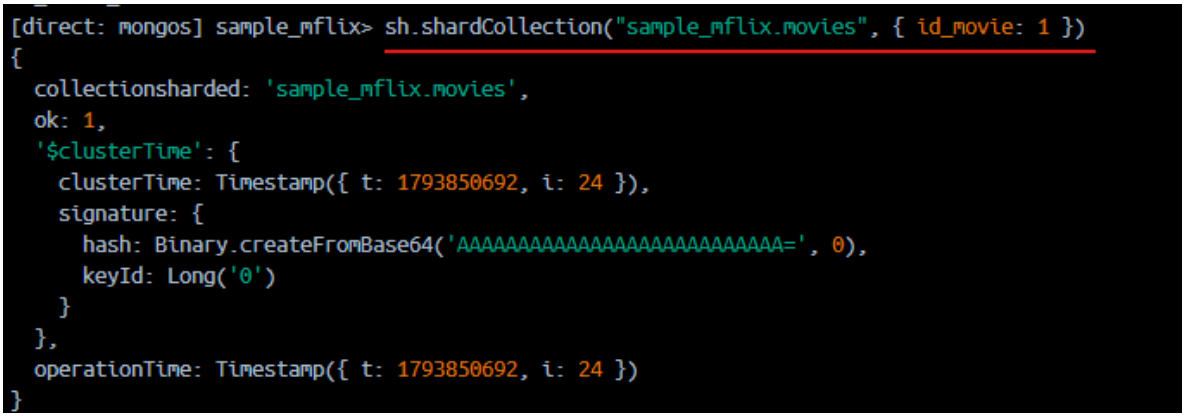
Indexar los documentos

Para poner sobre cada documento el id_movie, se ejecuto el siguiente script:

```
let counter = 1;
db.movies.find().forEach(function(doc) {
    db.movies.updateOne(
        { _id: doc._id },
        { $set: { id_movie: counter++ } }
    );
});
```

Definir el Shard Key

inicialmente se configuró un shard key de tipo *hashed*; sin embargo, la colección ya contenía datos al momento de shardearla, por lo que MongoDB creó un solo chunk que cubría todo el rango de hashes. Como el balanceador no mueve chunks hasta que este crece y se divide (al alcanzar el tamaño máximo, 64 o 128 MB), y el volumen de documentos era insuficiente para que eso ocurriera, toda la información permaneció en un único shard.

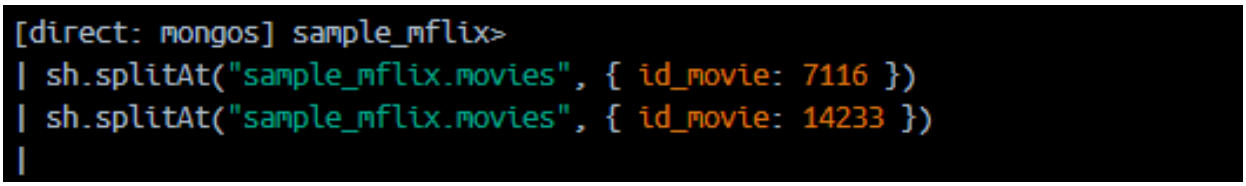


Antes de definir el Shard Key se habilito el particionamiento:

```
sh.enableSharding("sample_mflix")
db.movies.createIndex({ id_movie: 1 })
```

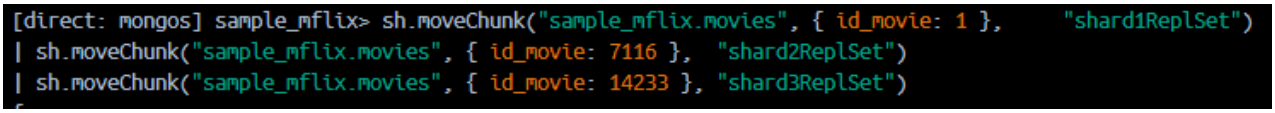
Balanceo de carga manual

Para distribuir los documentos de la forma más uniforme posible se tomó en cuenta que era 21349 documentos para dividirlos en tres partes. Con el método splitAt para cortar los chunks.



Mover los chunks

Una vez definidos los límites de cada chunk estos se movieron a cada shard.



Resultado

Los documentos quedaron distribuidos casi en proporciones iguales.

```
Totals
{
  data: '33.22MiB',
  docs: 35581,
  chunks: 3,
  'Shard shard1ReplSet': [
    '32.55 % data',
    '19.99 % docs in cluster',
    '1KiB avg obj size on shard'
  ],
  'Shard shard2ReplSet': [
    '34.52 % data',
    '20 % docs in cluster',
    '1KiB avg obj size on shard'
  ],
  'Shard shard3ReplSet': [
    '32.91 % data',
    '60 % docs in cluster',
    '1KiB avg obj size on shard'
  ]
}
```

Inconveniente

Si la base de datos sigue creciendo todos los nuevos documentos caerán en el último shard.

Solución

Al darnos cuenta de esto decidimos que era mejor buscar una manera de que se pudiera realizar con el Shard Key de tipo Hashed.

Creamos una copia de la base de datos

```
[direct: mongos] sample_mflix> db.movies.countDocuments()
21349
[direct: mongos] sample_mflix> db.movies.aggregate([{ $out: "movies_backup2" }])
```

Borramos la anterior base de datos

```
[direct: mongos] sample_mflix> db.movies.drop()
true
[direct: mongos] sample_mflix> db.movies.countDocuments();
0
```

Creamos el shardkey antes de insertar algún valor en la base de datos

```
[direct: mongos] sample_mflix> sh.shardCollection("sample_mflix.movies", { id_movie: "hashed" }, false, { numInitialChunks: 6 })
{
  collectionsharded: 'sample_mflix.movies',
  ok: 1,
  '$clusterTime': {
    clusterTime: Timestamp({ t: 1793861856, i: 35 }),
    signature: {
      hash: Binary.createFromBase64('AAAAAAAAAAAAAAAAAAAAAAAAAAA=', 0),
      keyId: Long('0')
    }
  },
  operationTime: Timestamp({ t: 1793861856, i: 35 })
}
```

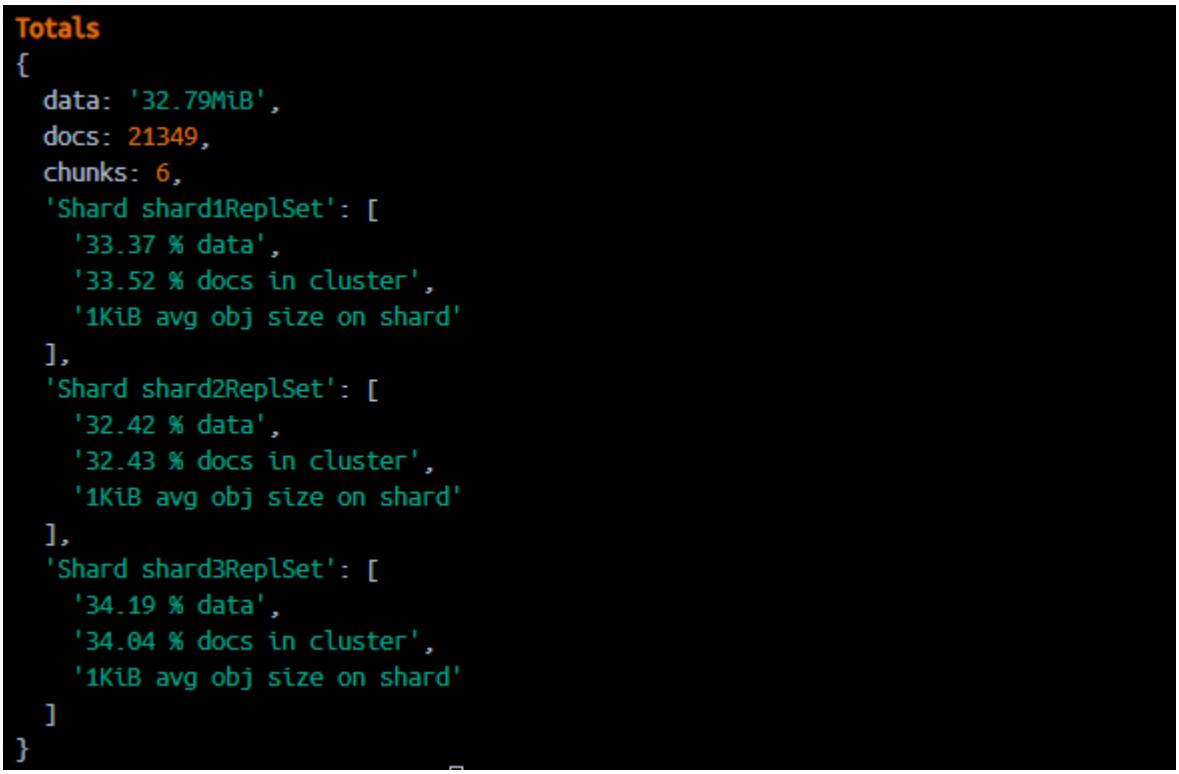
Cargamos la base de datos

La copia de la base de datos que creamos la utilizamos para cargar la nueva base de datos.

```
[direct: mongos] sample_mflix> let batch = [];
| db.movies_backup2.find().forEach(function(d) {
|   batch.push(d);
|   if (batch.length === 1000) { db.movies.insertMany(batch); batch = []; }
| });
| if (batch.length) db.movies.insertMany(batch);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('573a13a4f29313caabd112a9'),
    '1': ObjectId('573a13a4f29313caabd115b3'),
    '2': ObjectId('573a13a4f29313caabd11a92'),
    '3': ObjectId('573a13a4f29313caabd11e36'),
    '4': ObjectId('573a13a4f29313caabd1286b'),
    '5': ObjectId('573a13a4f29313caabd12a01'),
    '6': ObjectId('573a13a5f29313caabd13058'),
```

Resultado

Vemos como los documentos quedan distribuidos equitativamente entre todos los tres shards.



Conclusión

Definir la Sharding Key antes de insertar los documentos permite que MongoDB conozca desde el principio el criterio de particionamiento y pueda distribuir los datos conforme se incorporan a la colección. En cambio, si la Sharding Key se define después de haber insertado los documentos, estos pueden quedar inicialmente concentrados en un único shard, debido a que MongoDB no redistribuye automáticamente toda la información existente al establecer la clave (MongoDB crea inicialmente un único chunk con el valor por defecto de 128MB que abarca todo el rango de valores de la Sharding Key).